

DAGSmith: Dependency-Aware Rewriting for dbt-Style SQL Pipelines

Jie Liu

jiezzliu@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Lin Ma

linmacse@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Barzan Mozafari

mozafari@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

ABSTRACT

Modern analytics is increasingly organized as recurring SQL pipelines rather than isolated SQL statements. Tools such as dbt—which have gained extreme popularity in recent years—allow teams to write each transformation as SQL and make dependencies between transformations explicit, producing directed acyclic graphs (DAGs) with hundreds or thousands of interdependent SQL models. Existing techniques are not effective at optimizing these expensive pipelines. Traditional query optimizers and source-to-source query rewriters operate on one query at a time, while materialized-view selection and multi-query optimization address narrower forms of reuse. They do not directly exploit the pipeline-level information exposed by explicit dependencies: how intermediate results are consumed, which downstream outputs depend on each computation, where expensive work occurs relative to data reduction, which intermediate results are worth persisting, and how refresh schedules relate to the rate at which inputs change and outputs are consumed.

We introduce DAGSmith, to the best of our knowledge the first holistic dependency-aware source-to-source rewriting system for SQL pipeline DAGs. DAGSmith treats explicit dependencies as optimization signals. It analyzes each transformation together with its upstream inputs, downstream consumers, and position in the pipeline DAG, uses an LLM to propose pipeline-level refactorings, separates SQL generation and equivalence checking to reject unsafe rewrites, retunes persistence choices with a learned cost model, and selects a globally compatible set of rewrites that avoids conflicts across transformations. This enables dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization while keeping the pipeline executable by the same SQL engine and orchestration framework. Our extensive evaluation shows that, on the open-source Tuva dbt project, DAGSmith reduces elapsed time by **42.6%** and warehouse compute cost by **67.7%**; these reductions are **22.8%/83.0%** larger than materialization-only tuning and **98.1%/348.3%** larger than state-of-the-art single-query rewriting techniques.

1 INTRODUCTION

Rising SQL Complexity — SQL is no longer only a hand-written interface for individual analytical questions. It is increasingly the implementation language for recurring data products: governed dashboards, AI-ready feature tables, self-service BI datasets, and organization-wide reporting layers. At the same time, more SQL is produced by BI systems, code generators, and AI-assisted development tools rather than written from scratch. In fact, recent

studies report that 70% of analytics professionals use AI for code development [9], and that BI-generated SQL contain extremely complex scalar expressions and relational operator trees [20]. This has created a major efficiency gap between the complexity of the SQL queries generated and what today’s query optimizers are capable of effectively optimizing. Source-to-source query rewriting is the traditional way to narrow this gap: a rewriter transforms complex SQL into an equivalent form that the query optimizer is more likely to turn into an efficient plan. However, query rewriting has traditionally treated one SQL statement as the unit of optimization. Unfortunately, the effectiveness of this one-query-at-a-time approach is increasingly limited by a second trend: modern SQL is no longer produced and executed only as isolated statements, but as recurring transformation pipelines.

Rise of SQL Pipelines — Data teams are increasingly building large, recurring SQL pipelines. This growth is driven by complex business logic that must be computed repeatedly and reused consistently across governed dashboards, AI-ready feature tables, self-service BI datasets, compliance reports, and downstream applications. One of the most widely adopted tools for these workflows is *dbt* (the data build tool) [11], reported to be used by 50K+ data teams weekly across the world [7]. *dbt* lets teams write transformations as version-controlled SQL SELECT statements (**called “models” in dbt, which we use interchangeably in this paper**), declare dependencies among them, and compile the resulting graph into SQL jobs executed by a data warehouse. Figure 1 gives a toy example. A raw orders table feeds a cleaning transformation; the cleaned result feeds both a revenue table and a daily active-customer table. The SQL text explains each local transformation, while the edges explain how the transformations compose into a pipeline. This trend is much broader than *dbt*. SQLMesh [19], Dagster asset graphs [4], Airflow directed acyclic graphs [2], Prefect flows [15], Argo Workflows [3], Databricks Workflows [5], and medallion lakehouse pipelines [6] all represent recurring data transformations through explicit dependencies. In this paper, we target the SQL subset of this broader pattern, which we refer to as *SQL pipeline DAG*: a recurring directed acyclic graph whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node’s output.

Pipeline Optimization Gap — These pipeline DAGs often include hundreds or thousands of SQL transformations that reference one another. Their scale makes them difficult to optimize with techniques designed for a single SQL statement or a flat workload: inlining every referenced transformation into each downstream query would produce deeply nested SQL with tens or hundreds of layers whose size and complexity are well beyond what today’s

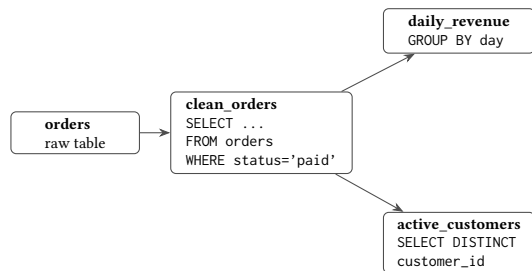


Figure 1: A toy SQL pipeline DAG. Nodes are SQL transformations or SQL-produced tables (called “models” in dbt); edges show which transformation consumes another transformation’s output.

optimizers can even tackle. In particular, existing *query rewriting* techniques, whether traditional rule-based systems [21] or modern synthesis- or LLM-based techniques [10, 13, 18], rewrite one SQL statement at a time. This scope is useful for improving a standalone query, but it is too narrow for optimizations whose correctness or benefit depends on the surrounding pipeline, such as removing statements no final output uses, reusing equivalent logic across separate transformations, or changing refresh frequency based on input changes and output consumption. *Multi-query optimization* techniques [16, 17] consider multiple queries together, but their classical goal is to share common scans, joins, or intermediate results across a batch of submitted queries. They do not generally rewrite a SQL pipeline by adding, removing, splitting, or merging transformations based on downstream demand and refresh behavior. *Materialized-view selection* [1, 12] chooses intermediate query results to precompute and rewrites future queries to read those precomputed results. This improves reuse when the relevant computation has already been identified, but it does not by itself determine that a pipeline should drop unused work, factor repeated logic written in different forms, move expensive operations after data reduction, or refresh different parts of the pipeline at different rates. Lastly, *Workflow schedulers* [2–4, 15] know dependencies, but usually optimize execution order, failure recovery, retries, or resource allocation rather than the SQL semantics of the transformations themselves.

Dependencies as Signals — Our key insight is that explicit dependencies are not merely execution constraints; they are signals that can be leveraged for optimization. In particular, a SQL pipeline DAG exposes six kinds of information that are hidden or much harder to recover when queries are optimized independently: (1) **Edge semantics**: producer-consumer edges reveal how one SQL transformation’s output is used by later transformations; (2) **Cross-graph redundancy**: similar business logic can appear in distant parts of the graph, even when SQL text differs; (3) **Downstream demand**: downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs; (4) **Operator placement**: the graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations; (5) **Rewrite-persistence coupling**: reuse patterns and fanout reveal when a rewritten SQL (model) should be materialized as a stored result, and when it should instead be inlined or recomputed by downstream transformations; and (6) **Refresh patterns**:

schedules and source update rates reveal how often inputs change and outputs are consumed.

Our Approach — In this paper, we introduce DAGSmith, the first (to the best of our knowledge) holistic dependency-aware rewriting system for SQL pipeline DAGs. DAGSmith exploits the exposed dependency structure to perform six classes of optimization: (1) **dependency-edge simplification**: remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs; (2) **non-local semantic reuse**: factor repeated work into a shared computation across non-adjacent transformations; (3) **downstream-aware pruning**: remove work that is provably irrelevant to every final output; (4) **pipeline-aware work placement**: move expensive work after data-reducing steps as long as final outputs are not impacted; (5) **rewrite-materialization co-optimization**: decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile; and (6) **frequency-aware optimization**: align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

DAGSmith first analyzes compiled SQL and dependency edges to identify regions likely to contain these opportunities. It then leverages modern LLMs to reason over the relevant subgraph and propose concrete refactorings, but separates proposal, criticism, SQL generation, and equivalence checking so unsafe or counterproductive rewrites can be rejected before adoption. Adopted candidates are not evaluated in isolation: DAGSmith retunes persistence choices for each candidate using a learned cost model and an iterative local linearization procedure, then solves a global selection problem to choose a non-conflicting subset of rewrites. Finally, frequency information from source update rates and output demand reweights cost estimates, detects over-scheduled work, and generates splitting candidates when slow-changing logic is embedded in fast-refreshing transformations. The result is a rewritten equivalent SQL pipeline that remains executable by the same SQL engine and orchestration framework, but is significantly more performant.

Contributions — This paper makes the following contributions:

- (1) We formulate dependency-aware source-to-source rewriting for SQL pipeline DAGs. We present the first holistic approach to this problem (to the best of our knowledge) by identifying the pipeline-level signals exposed by explicit dependencies and the rewriting opportunities they enable: dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization.
- (2) We present the design of DAGSmith, which combines structural graph analysis, LLM-guided refactoring, separated proposal, criticism, SQL generation, and equivalence checking, learned cost modeling, persistence tuning, frequency-aware scheduling, and global candidate selection.
- (3) We conduct a comprehensive evaluation, showing that DAGSmith reduces the end-to-end completion time and monetary cost by **XX%** and **WW%** over the original pipeline, by **YY%** and **WW%** over materialization-only tuning, and by **ZZ%** and **WW%** over single-query rewriting baselines, respectively.

Table 1: Optimization signals exposed by explicit SQL pipeline dependencies.

#	What explicit dependencies expose	How DAGSmith leverages them for optimization
1	Producer-consumer edges reveal how one SQL transformation’s output is used by later transformations.	Dependency-edge simplification: remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs.
2	Similar business logic can appear in distant parts of the graph, even when SQL text differs.	Non-local semantic reuse: factor repeated work into a shared computation across non-adjacent transformations.
3	Downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs.	Downstream-aware pruning: remove work that is provably irrelevant to every final output.
4	The graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations.	Pipeline-aware work placement: move expensive work after data-reducing steps as long as final outputs are not impacted.
5	Reuse patterns and fanout reveal when a rewritten model should be materialized as a stored result, and when it should instead be inlined or recomputed by downstream transformations.	Rewrite-materialization co-optimization: decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile.
6	Schedules and source update rates reveal how often inputs change and outputs are consumed.	Frequency-aware optimization: align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

The rest of the paper is organized as follows. Section 2 defines the setting and illustrates the opportunity space. Section 3 presents the design of DAGSmith. Section 4 outlines the evaluation plan, Section 5 discusses prior work, and Section 6 concludes.

2 PROBLEM SETTING

Before presenting our approach, we define the optimization setting that DAGSmith targets and illustrate, through three examples drawn from or inspired by real-world analytics pipelines, the kinds of dependency-aware optimizations that this setting enables but that fall outside the reach of existing techniques.

2.1 Background: SQL Pipeline DAGs

A *SQL pipeline DAG* is a directed acyclic graph $G = (V, E)$ whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node’s output. We use “DAG” in the standard sense: the dependency graph is directed, and cycles are disallowed so the transformations can be evaluated in dependency order. A node may be implemented as a dbt *model* (dbt’s term for a SQL transformation that produces a table or view), a notebook cell that issues SQL, a warehouse task, or a table produced by an orchestration framework. The graph may be declared directly, as in Airflow [2] and Argo [3], inferred from dataflow, as in Prefect [15], or induced by model references, as in dbt [8] and SQLMesh [19]. In asset-oriented systems such as Dagster [4], the same idea appears as an asset graph: nodes are persistent data assets and edges describe how one asset is derived from another.

dbt is the running example in this paper because its SQL transformation boundaries and dependency declarations make the setting especially explicit. However, the optimization problem is broader. Databricks Workflows [5] and Lakeflow Jobs [5] expose task dependencies as a job DAG; Databricks’ medallion architecture [6] encourages multi-hop transformations from bronze to silver to gold tables; Luigi [14] and the systems above all represent recurring data work through dependency graphs. DAGSmith targets the subset of these pipelines where the node logic is available as SQL, or can be

compiled to SQL, so that the optimizer can reason about relational semantics rather than only task ordering.

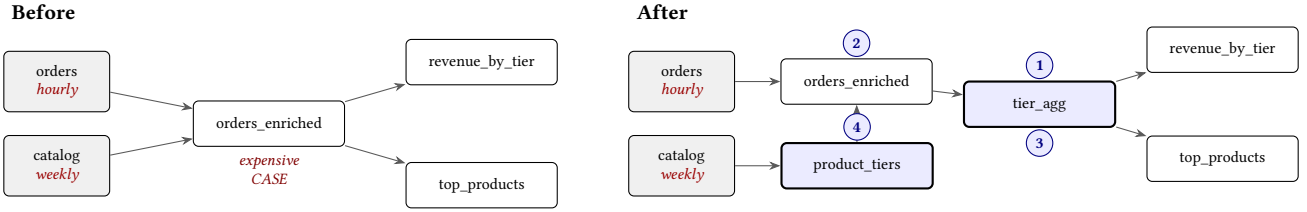
2.2 Motivating Examples

This setting differs from traditional query optimization in six ways that matter for our problem. Figure 2 previews several of the changes these differences make possible.

(1) Dependency-edge simplification. The graph identifies explicit boundaries between SQL transformations: which output is produced at one node, which later nodes read it, and which final outputs must remain unchanged. These boundaries are optimization choices, not just execution constraints. A dependency-aware optimizer can therefore bypass an intermediate node, inline part of a producer, pull a predicate from a consumer into a producer, or replace a dependency on one node with an equivalent expression over another node, as long as the final pipeline outputs are preserved. This is stronger than expanding a view inside one query: the optimizer can change how SQL is divided across transformations, not merely optimize the compiled text of one statement.

(2) Non-local semantic reuse. The graph exposes similar business logic that appears in different parts of the pipeline, often under different names or SQL structure. A dependency-aware optimizer can factor repeated work into a shared computation and rewrite consumers to use that shared result. This is not limited to syntactic common subexpressions: two transformations may compute the same logical quantity using different predicates, aliases, joins, or intermediate names.

(3) Downstream-aware pruning. Downstream transformations reveal which columns, joins, or intermediate results are never consumed by any final output. A dependency-aware optimizer can remove such work safely because the graph proves that no later transformation depends on it. A one-query rewriter can remove locally dead expressions inside one statement, but it cannot know



- (1) *Dependency-edge simplification.* Producer-consumer edges identify SQL boundaries that can be replaced, bypassed, or shifted when an equivalent relationship exists across the boundary.
- (2) *Non-local semantic reuse.* Similar logic in separate parts of the graph can be factored into a shared computation even when the SQL text differs.
- (3) *Rewrite-materialization co-optimization.* A shared intermediate is useful only under the right persistence choice; rewriting and persistence must be optimized together.
- (4) *Frequency-aware optimization.* The expensive classification depends only on weekly data but is recomputed inside an hourly transformation; splitting it onto the slow cadence requires a pipeline rewrite invisible to per-query cost.

Figure 2: A compact example of optimization opportunities exposed by an explicit SQL pipeline DAG. Once dependencies are visible, the optimizer can simplify dependency edges, factor repeated logic, co-optimize rewriting with persistence, and split slow-changing logic off fast refresh paths.

whether an intermediate result produced for the pipeline is actually read later.

(4) Pipeline-aware work placement. The graph shows where expensive operations occur relative to filters, projections, aggregations, and joins. A dependency-aware optimizer can move expensive work after data-reducing steps as long as final outputs are not impacted. This opportunity is pipeline-level because the data-reducing step and the expensive operation may be separated across transformation boundaries.

(5) Rewrite-materialization co-optimization. The graph shows which rewritten results would be reused and where recomputation would occur if they are not persisted. A dependency-aware optimizer can choose rewrites and persistence choices together, since a rewrite can create new persistence opportunities and persistence can determine whether a rewrite helps. For example, a shared transformation may reduce cost when written once as a table but increase cost when inlined as a view under several consumers.

(6) Frequency-aware optimization. Schedules and source update rates reveal how often inputs change and outputs are consumed. A dependency-aware optimizer can align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers. In cloud warehouses, these rates translate directly into cost and latency objectives: if a weekly source feeds an hourly transformation, any computation that depends only on the weekly source is being repeated unnecessarily; if an output is consumed daily, hourly refresh may be waste even when the SQL statement itself is individually optimized.

Together, these properties define a distinct optimization setting. The optimizer is reasoning not about one query, nor merely about an unordered set of independent queries, but about a recurring SQL pipeline program whose graph structure, final outputs, persistence choices, and refresh behavior interact.

We now illustrate three representative classes that DAGSmith discovers. Each requires the optimizer to read and reason about SQL definitions across transformation boundaries within the DAG. The

SQL in each example is simplified for presentation; domain-specific details are abstracted away.

Example 1: Non-local semantic reuse. A pipeline contains seven classifier transformations that each tag orders with a service category (dialysis, psychiatric, and five others). The input is a claims table in which each row represents one item of an order, so a single order has many rows. Each classifier scans claims, keeps rows whose codes match its category’s code list, and returns the distinct order_ids of those rows. The seven classifiers have *different* SQL: different predicates, different code vocabularies, and in some cases different join structures. Two representative classifiers are shown on the left of Figure 3.

Before: 7 classifiers (2 shown)

```
1 -- dialysis_classifier
2 SELECT DISTINCT order_id
3 FROM {{ ref('claims') }}
4 WHERE bill_code LIKE '72%'
5 OR taxonomy IN (set_dia_tax)
6 OR diagnosis IN (set_dia_dx)
7
8 -- psychiatric_classifier
9 SELECT DISTINCT order_id
10 FROM {{ ref('claims') }}
11 WHERE revenue IN (set_psy_rev)
12 OR diagnosis BETWEEN
13 '290' AND '319'
```

After: precomputation+lookups

```
1 -- order_flags (new)
2 SELECT order_id,
3 MAX(CASE WHEN bill_code LIKE '72%'
4 OR taxonomy IN (set_dia_tax)
5 OR diagnosis IN (set_dia_dx)
6 THEN 1 ELSE 0 END) AS f_dia,
7 MAX(CASE WHEN ...
8 THEN 1 ELSE 0 END) AS f_psy,
9 ... -- 7 categories
10 FROM {{ ref('claims') }}
11 GROUP BY order_id
12
13 -- dialysis_classifier (rewritten)
14 SELECT order_id
15 FROM {{ ref('order_flags') }}
16 WHERE f_dia = 1
```

Figure 3: Non-local semantic reuse. Seven classifiers with no syntactic overlap perform the same semantic operation on a shared parent. DAGSmith synthesizes a new shared transformation that pre-aggregates all classifier flags in one pass.

The seven classifiers share no common subexpression, yet they all perform the same *semantic operation*: for each order, decide whether any of its rows match a category-specific predicate. DAGSmith recognizes this cross-transformation pattern and factors it. It synthesizes a new intermediate transformation `order_flags` that, in a single pass over `claims`, computes one boolean flag per category by aggregating each category’s predicate with `MAX(CASE WHEN ... THEN 1 ELSE 0 END)` grouped by `order_id`. Each original

classifier is then rewritten as a lightweight lookup: select the orders whose corresponding flag in `order_flags` is 1. Seven full scans and seven independent grouping passes over `claims` are replaced with one pre-aggregation pass and seven inexpensive boolean filters.

This optimization requires three capabilities that go beyond traditional techniques. First, the optimizer must recognize that seven syntactically dissimilar transformations perform the same semantic operation, which syntactic subexpression matching, the basis of multi-query optimization (MQO) [17] and common subexpression elimination, cannot do. Second, it must *invent* a new intermediate transformation absent from the original project; materialized view recommendation tools [1] synthesize candidate views, but they too rely on syntactic subexpression mining, of which the seven classifiers share none. Third, it must rewrite the dependent transformations to consume the new intermediate while preserving end-to-end equivalence, a multi-transformation restructuring that single-query rewrite systems [18, 21] do not attempt.

Example 2: Dependency-edge simplification. The same project contains a classifier nursing that filters for skilled-nursing claims by facility code, then performs an anti-join against a sibling transformation equipment to exclude equipment-related claims (Figure 4, left). The anti-join forces the engine to produce equipment, build a hash table, and probe it for every row of `claims`; it also introduces a dependency edge that serializes the two transformations.

Before: anti-join	After: predicate
<pre> 1 -- nursing 2 SELECT * 3 FROM {{ ref('claims') }} c 4 WHERE c.facility_code 5 IN ('31','32') 6 AND NOT EXISTS (7 SELECT 1 8 FROM {{ ref('equipment') }} e 9 WHERE e.order_id = c.order_id 10 AND e.line_id = c.line_id) 11 12 -- equipment (just a filter) 13 SELECT * 14 FROM {{ ref('claims') }} 15 WHERE category = 'equipment' </pre>	<pre> 1 -- nursing (rewritten) 2 SELECT * 3 FROM {{ ref('claims') }} c 4 WHERE c.facility_code 5 IN ('31','32') 6 AND (c.category IS NULL 7 OR c.category <> 'equipment') 8 9 10 -- equipment is removed if no 11 -- other transformation reads it; 12 -- otherwise its definition 13 -- is unchanged. </pre>

Figure 4: Dependency-edge simplification. By reading the SQL of the sibling transformation equipment, DAGSmith determines that the anti-join is equivalent to a scalar predicate on a shared parent. The hash anti-join, the auxiliary scan, and the dependency edge are all eliminated.

By reading the SQL definition of equipment, DAGSmith discovers that it computes nothing more than a single-predicate filter on a shared parent. The anti-join is therefore equivalent to an inline scalar predicate on `claims`, as shown on the right of the figure: the hash anti-join, the auxiliary scan, and the cross-transformation dependency are all eliminated. The same reasoning generalizes to every sibling classifier that uses equipment as an exclusion list, compounding the benefit across the pipeline.

The equivalence enabling this rewrite is not declared anywhere; it is *embedded in the SQL of another transformation file*. A traditional engine sees equipment either as a base table to be probed or as a view to be expanded into the local query, neither of which performs the cross-transformation analysis needed to eliminate

the join entirely. We acknowledge a tradeoff: the rewrite couples nursing to the current definition of equipment, so a future change to that definition would require updating the rewritten consumers. In practice, stable upstream transformations such as equipment change rarely, and DAGSmith re-evaluates the project after each developer change, so the cost of re-deriving the rewrite is amortized over many executions.

Example 3: Frequency-aware optimization. Different sources in a SQL pipeline DAG update at different rates, and the structure of the DAG determines how much computation must be repeated at each high-frequency refresh. Consider a transformation that joins fast-changing transactional data with slow-changing reference data and applies expensive classification logic to the slow-changing side (Figure 5, left).

Before: hourly classification	After: split by frequency
<pre> 1 -- order_summary (hourly) 2 SELECT o.order_id, o.amount, 3 CASE 4 WHEN p.code IN (set_A) THEN 'A' 5 WHEN p.code IN (set_B) THEN 'B' 6 ... 7 END AS category 8 FROM {{ ref('orders') }} o 9 JOIN {{ ref('catalog') }} p 10 ON o.product_id = p.product_id 11 12 -- orders: hourly 13 -- catalog: weekly </pre>	<pre> 1 -- product_categories (weekly) 2 SELECT product_id, 3 CASE 4 WHEN code IN (set_A) THEN 'A' 5 WHEN code IN (set_B) THEN 'B' 6 ... 7 END AS category 8 FROM {{ ref('catalog') }} 9 10 -- order_summary (hourly) 11 SELECT o.order_id, o.amount, 12 pc.category 13 FROM {{ ref('orders') }} o 14 JOIN {{ ref('product_categories') }} pc 15 USING (product_id) </pre>

Figure 5: Frequency-aware optimization. Splitting the transformation along the frequency boundary moves expensive classification onto a weekly cadence; the hourly transformation becomes a lightweight join.

Because orders changes hourly, `order_summary` re-executes hourly, and the multi-branch classification over catalog is recomputed on every hourly refresh, despite its inputs changing only weekly. DAGSmith traces the update frequency from each source through the DAG, identifies that this transformation has *mixed-frequency inputs*, and splits it along the frequency boundary. The expensive classification is extracted into a new slow-frequency transformation, and the original transformation becomes a lightweight equi-join. When this pattern recurs across many consumers of the same slow-changing reference data, the savings compound: each hourly refresh avoids redundant recomputation of logic whose inputs have not changed.

This optimization is invisible to optimizers that treat cost as a fixed per-query quantity: the benefit of splitting the transformation is realized only when the two halves are known to execute at different rates. Materialized view selection considers update cost but does not restructure the DAG to align computation with update frequency, and traditional query rewriters have no notion of refresh schedule at all.

2.3 Why Local Rewriting Falls Short

The three examples above are not isolated curiosities; they illustrate a general capability gap. Single-query rewrite systems [18, 21] operate within the boundary of one query and cannot restructure

across transformation files. Multi-query optimization and common subexpression elimination [17] share intermediate computations across queries but rely on syntactic matching, so they miss the cross-transformation semantic equivalence in Example 1. Materialized view selection [1] chooses among existing view candidates and treats update cost as a static parameter rather than as a structural driver of DAG design (Example 3). Code clone detection and lineage tools can identify structural similarity or trace data provenance, but they do not perform the optimization itself.

The gap is not that any one of these techniques is poorly designed; it is that none of them targets the setting we are concerned with. Optimizing a SQL pipeline DAG requires an optimizer that can combine the six capabilities above: use producer and consumer context, find semantic reuse beyond syntactic overlap, prune work that no downstream output consumes, move expensive work after data-reducing steps, evaluate rewrites together with persistence choices, and account for recurring refresh behavior. The remainder of this paper presents an approach that integrates these capabilities into a coherent optimization framework.

3 APPROACH

lin: Okay in general, I think this section has way too many implementation details and less intuitive lessons. The purpose of a paper is not to document every details of the implementation so that others will recreate every single step. You'll have your code released and they can reference your code anyway. Rather, the purpose is to convey what are the key technical challenges to solve this problem, (intuitively) what are our solutions to solve these challenges and why we choose these solutions, and what lessons did we learn. Implementation details are helpful only if we have the space and if that supports our argument for the solution. But having too many can be distracting. I also think having 5 subsections (components) are a bit too many. I would suggest having only 3-4 subsections. (I think earlier you wrote there are 3 optimization dimensions? That might be a good breakdown for the subsections. It seems that in the ablation study you only categorize three optimizations anyway.) And within each subsection, you start with what technical challenges you're going to solve (and why such challenges exist in our problem), and intuitively how your solution works (and why it solves the challenges). A figure illustrating the solution would be a plus. Then, you go into a little bit of details to just have the audience have a slightly deeper understanding. You can take a look at other papers (or even your previous papers) to see how long this should be. Readers/Reviewers won't appreciate if you add too many details and they couldn't easily get the big picture.

DAGSmith optimizes a SQL pipeline DAG along three interacting dimensions: graph refactoring, materialization, and refresh frequency. lin: I know Barzan is still reorganizing sections 0-2, but since we have three dimensions here, we know just condense the earlier six opportunities to three, and we have an optimization for each of the opportunity? We can probably connect each of the example in Section 2 to one dimension here as well. Wouldn't this be much more succinct and saving lots of space? These are the same three dimensions our evaluation ablates, and each takes up a group of the opportunities from Section 2.2: refactoring restructures the

DAG to share or remove work (opportunities (1)–(4)), materialization decides which results are persisted and composes overlapping rewrites (opportunity (5) and global rewrite composition), and frequency aligns each node's execution rate with how often its inputs change (opportunity (6)). They cannot be decided independently.

lin: I think we need to reference Figure 5 at the beginning of this paragraph DAGSmith realizes these dimensions in the pipeline of Figure 6. Cheap structural analyses first scan the whole DAG and rank model groups by a fingerprint-based score that estimates the benefit of refactoring them jointly (Section 3.1). The top-ranked groups are passed to an LLM, which reasons over each region and proposes one or more concrete refactorings (Section 3.2). The structural analyses decide where in the DAG to refactor; the LLM decides what refactoring to perform. lin: Since we mentioned LLM, I think we need to add one sentence to talk about the correctness guarantee. Otherwise, I think that would be the first question that the reviewer raises. Every refactoring is adopted only after it passes equivalence checks that the LLM writes for itself and that are tested against the project's real data. A refactoring's benefit depends on how its nodes are materialized, so DAGSmith tunes each candidate's materialization with a learned cost model and selects a compatible subset to adopt (Section 3.3). The frequency dimension cuts across these stages (Section 3.4): it reweights the cost-driven decisions, removes refreshes that recompute unchanged data, and contributes structural splitting candidates to the first stage.

3.1 Candidate Identification

lin: I don't think we have established why we want to use LLM for refactoring yet. So I think either you have to establish that here, or just talk about refactoring is costly in general, without specifically emphasizing LLMs. Candidate identification finds where in the DAG the refactoring opportunities (1)–(4) of Section 2.2 are worth pursuing. Refactoring reasons jointly about a region of interdependent models, and a rewrite spanning many of them is hard to keep correct; DAGSmith therefore works on small, bounded regions rather than the whole project. Because most regions are not worth refactoring, it scans the whole DAG with cheap structural analyses that rank regions by likely benefit, reserving the expensive refactoring step (§3.2) for the few that pay off.

lin: Before getting into all the details below, I think it would be good to have one paragraph to discuss intuitively how your analysis works and what it achieves (if you can pair that with a figure, that's even better). The rest of this subsection is just implementation details. If intuitively the method makes sense, we would be good. We can either keep the details below or just condense them if we're short on space. At its core the analysis must decide when two models do the same expensive work, and the obvious tests miss real cases. Matching SQL text fails on any rephrasing. Matching plan structure is stronger, since multi-query optimization and view-matching already handle join order, predicate order, and aliasing, but it is keyed on the shape of one compiled query and treats a referenced model as an opaque input, so it misses work shared across model boundaries. In Figure 7, Models A and B compute the same join, but Model A filters its input inline while Model B reads the already-filtered input from a sibling model it references, so on the compiled SQL a plan-subtree matcher sees two different

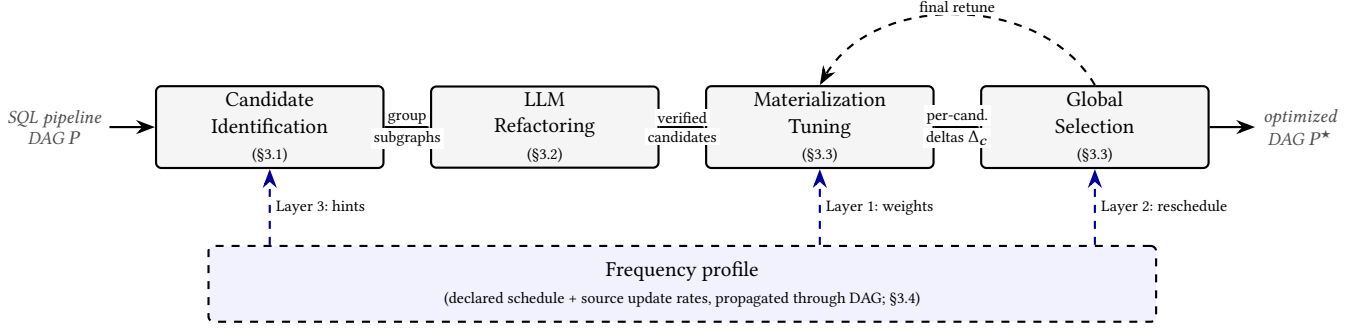


Figure 6: End-to-end pipeline of DAGSmith. The top row shows the four main stages and the artifacts that flow between them; after global selection, a final materialization pass retunes the combined DAG. The dashed band underneath is the frequency-aware overlay (§3.4): a single frequency profile feeds three of the four stages, contributing structural splitting hints, cost weights for materialization tuning, and schedule corrections at selection.

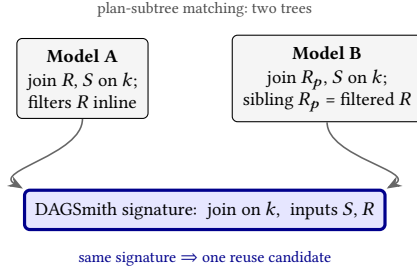


Figure 7: Candidate identification matches on the operation a model computes, not on its plan shape. Models A and B compute the same join on k , but Model A filters R inline while Model B reads the already-filtered R from a sibling model R_p ; on the compiled SQL the two are different trees over different tables, while DAGSmith resolves the reference and reduces both to one signature and a single reuse candidate.

trees over different tables. DAGSmith instead matches on the operation each model computes, resolving references so that a model boundary does not hide shared work, and reduces the two to a single reuse candidate that the LLM and an equivalence check later confirm is safe to share. Two analyses work this way: *reuse* finds repeated expensive work to share (opportunity (2)), and *prune* finds work that no downstream model consumes or that runs too early (opportunities (1), (3), and (4)). Only the highest-scoring regions go to the LLM.

Dataflow representation. Both analyses run on a *dataflow representation* rather than on SQL text: DAGSmith compiles each model into a tree of typed operators and resolves every column and `ref()` back through the models that produced it. Figure 7 shows why this is needed. Model B reads the sibling R_p , so at the SQL level its input is an opaque table name; resolving the reference substitutes R_p ’s definition, the filtered R , and reconstructs the join B actually computes over R and S . Model A reaches the same operation by filtering R inline. The two resolved trees coincide even though the `ref()` boundary hides the correspondence in the source, and matching on this representation is what lets the analyses compare models by

what they compute rather than by how the SQL is split across the DAG.

Reuse analysis. Reuse analysis summarizes each model’s dataflow representation by a *signature*: its expensive operations (joins and group-by aggregations), the keys they operate on, and their resolved inputs. Models whose signatures match are grouped into one candidate and scored by the cost of the shared operation and the number of models that share it. A high score is a *targeting signal*, not an instruction to materialize a shared subexpression; it marks where the LLM should look.

Prune analysis. Prune analysis targets joins in two forms. A *redundant* join produces a result no downstream model consumes, confirmed by tracing column lineage from the leaf models, and can be removed outright or, when it is a semi-join whose filter is already derivable from the probed relation, collapsed to a predicate. A *mistimed* join is correct but computed too early, inflating an expensive downstream operation whose result does not depend on it; pushing it past that operation avoids carrying its product through the work. What sets both apart from local dead-code elimination is scope: whether a join’s result is used, or used too soon, is a fact about the rest of the pipeline that a single-query rewriter cannot see.

From candidates to subgraphs. The two analyses produce a ranked stream of candidate model groups. For each, DAGSmith extracts a subgraph of the candidate models plus a bounded neighborhood of upstream and downstream context, enough for the next stage to reason about correctness; groups whose neighborhoods exceed the bound are skipped. The next subsection describes how the LLM turns these subgraphs into concrete refactorings.

3.2 LLM Refactoring

lin: Well, why do we want to use an LLM to do this in the first place? We either have to establish this earlier or justify it here. This stage produces the refactorings that realize opportunities (1)–(4) of Section 2.2. As motivated in the overview, producing these cross-boundary, semantic rewrites is a reasoning task rather than a pattern-matching one, which is why DAGSmith uses an LLM rather than a fixed rule set. But asking an LLM to rewrite SQL that will run

against a production warehouse raises three problems: the rewrite may be *slower* than the original, may be *incorrect*, or may be *too expensive to generate* when a region contains many models. The challenge for this stage is to obtain useful rewrites while controlling all three risks.

lin: Similarly, I think one paragraph to discuss intuitively how your approach works and what it achieves (better with a figure) can be valuable. The details can be reduce. DAGSmith controls these risks by refusing to treat rewriting as a single LLM call. It splits the work into four roles, each a separate call with one job: a *proposer* reads the region and lists candidate optimizations, an adversarial *critic* rejects proposals that would slow the pipeline or silently change its results, a *rewriter* turns the survivors into concrete SQL, and a *verifier* tests each rewrite against the warehouse’s real data before it is trusted. Separating proposal from criticism keeps a single prompt from having to be both creative and skeptical, and grounding acceptance in real data rather than in the model’s own assurance is what makes adopting LLM-written SQL safe.

Proposal. The first call ingests the subgraph’s SQL and dependency edges and produces a domain-level reading of the region: a short semantic summary of each model and the end-to-end business question the subgraph answers. This reading is what lets later stages recognize equivalent computations written in dissimilar SQL. Its output is a list of *opportunities*, each naming the work it would remove or shrink and the mechanism by which it would do so; opportunities are proposals, not yet rewrites.

Filtering. A second call acts as an adversarial critic over the proposals. The separation is deliberate: a single LLM asked at once to propose and to doubt does neither well, with proposal mode dominating. The critic catches both slowdowns and correctness traps. A slowdown is, for instance, a proposal that looks like reuse-driven savings but adds a write and a scan without reducing downstream work. A correctness trap is shown in Figure 8: the proposal drops a join by reading a tag from a sibling that has been deduplicated to one row per item, silently discarding the secondary tags the original join admitted. The critic returns one of three verdicts, *approve*, *reject*, or *conditional*; a conditional verdict records a data property the rewrite must hold to, which is carried into the next stage.

Original: join admits all tags	Proposal: drop join, read tag
<pre> 1 SELECT i.item_id, 2 MAX(CASE WHEN t.tag='A' 3 THEN 1 ELSE 0 END) AS flag_a 4 FROM ((ref('items'))) i 5 JOIN ((ref('item_tags'))) t 6 USING (item_id) 7 GROUP BY i.item_id </pre>	<pre> 1 SELECT s.item_id, 2 CASE WHEN s.tag='A' 3 THEN 1 ELSE 0 END AS flag_a 4 FROM ((ref('items_staged'))) s 5 -- items_staged keeps one tag/item; 6 -- secondary tags are silently lost </pre>

Figure 8: Correctness trap rejected by the proposal critic. The original aggregates over all tags per item; the proposal reads from a sibling that has been deduplicated to one tag per item, and silently drops the rest.

Rewriting. The opportunities that survive filtering, together with the analysis and the SQL of the affected nodes, go to a third call that produces concrete refactored SQL: it may rewrite nodes, mark nodes as removed, or introduce new shared nodes. Conditional opportunities carry their recorded constraints into this prompt.

A node with consumers outside the subgraph may be rewritten internally but must produce identical output rows, since its contract with those consumers is fixed.

Equivalence verification. A rewrite that passed filtering can still change the data, especially when one bundles several opportunities and a trap rejected on its own reappears nested inside a larger change: the staged-input shortcut of Figure 8, for instance, can resurface inside a consolidated aggregation the critic approved on other grounds. DAGSmith therefore treats equivalence as an empirical question against real data. A fourth call, the *correctness critic*, states for each adopted opportunity the data invariant that must hold for the rewrite to be safe and renders it as a SQL diagnostic that returns zero rows when the invariant holds and rows when it is violated. The diagnostics run against the target warehouse under a per-query byte cap; any returned row is a counterexample drawn from real data. Failed invariants are appended to the rewriting prompt, and the LLM must either find a distinctly different strategy that respects them or restore the original logic. The loop ends when the rewrite passes all diagnostics, the LLM reports no further distinct strategy, or a per-group attempt budget is exhausted, in which case the group reverts to its original SQL and contributes no candidate.

Model families and propagation. A subgraph often contains a *family* of near-duplicate models that apply the same operation to different data subsets, and running the full four-call sequence on each would be wasteful. DAGSmith instead runs it once on a representative and propagates the accepted rewrite to the rest with a lighter per-member call. Where a project has no family structure, the sequence runs over every model unchanged.

3.3 Materialization Tuning

This stage takes up two opportunities from Section 2.2: rewrite-materialization coupling (5) and global rewrite composition. A refactoring has no benefit that can be read off in isolation. Whether it helps depends on which of its nodes are stored as tables and which are inlined as views, and the best choice differs from one candidate to the next; a candidate must therefore be scored together with its own best materialization, not in the abstract. A second challenge follows once candidates are scored: they overlap, and must be composed into a compatible set rather than adopted all at once. This subsection addresses both, tuning one candidate’s materialization and then selecting a compatible set.

Tuning one candidate. Finding a candidate’s best materialization is hard because a node’s materialization is not a local choice. A table acts as a barrier: it is built once, and its consumers read its stored rows. A view does not; a query that reads a view re-executes the view’s SQL, expanding recursively through any parent that is itself a view until it reaches a table or source. Turning a node from table to view therefore removes its build cost but pushes its computation, along with the chain of views above it, into every query that reads it, once per consumer execution. A node’s cost is set by a window of the DAG around it, and because those windows overlap, the choices are coupled: the marginal value of one decision depends on the rest of the assignment. Total cost is not a sum of per-node costs, which rules out tuning node by node, and the joint space is far too large to enumerate.

Two different costs stand in the way, and DAGSmith addresses them separately. The first is evaluation: scoring a configuration would normally mean executing it. A learned cost model, trained on instrumented runs of the original pipeline, predicts the cardinality and slot time of every node under a given assignment, which prices a whole configuration without running it; the new nodes a refactoring introduces, having no measurements, are predicted the same way from their SQL and their parents’ cardinality.

The second is search. DAGSmith navigates the space with **iterative local linearization**, a successive-approximation scheme that replaces the true nonlinear cost with a local linear surrogate, solves it, and re-linearizes at the result. Concretely, at iteration k , holding the current assignment $t^{(k)}$ fixed, DAGSmith uses the cost model to measure two local quantities: a baseline build cost $B_j^{(k)}$ for each node j (its direct parents forced to tables) and an edge-local delta $U_{ij}^{(k)}$, the extra cost at j when only parent i is inlined as a view. Treating these as constants for the iteration, it solves a small ILP for $t^{(k+1)}$:

$$\min_{t,y} \sum_{j \in V} t_j B_j^{(k)} + \sum_{i \rightarrow j \in E} y_{ij} U_{ij}^{(k)}, \quad \text{s.t.} \quad y_{ij} = (1 - t_i) \wedge t_j,$$

where y_{ij} is 1 exactly when node j is built and reads a view-parent i , and a trust-region cap on flips keeps each step where the coefficients hold. Because flipping a node changes the cardinalities its descendants see, B and U are recomputed after each move and the step repeated until the assignment stops changing; an effect the current coefficients miss, such as the cost of also flipping a sibling, then surfaces in the next round. The loop converges to a local optimum rather than a global one. This is safe because the unrefactored DAG is itself scored as a baseline candidate: a candidate that lands on a poor assignment loses to it and is not adopted; imperfect search costs a missed opportunity, never a regression below the original.

A candidate’s *delta* is the cost of the original under its best materialization minus the candidate’s under its own: the reduction the refactoring delivers when adopted alone.

Selecting a compatible set. Each delta is measured as if its candidate were adopted alone, but the candidates cannot simply be unioned. Two candidates conflict when they rewrite, create, or remove any of the same models, and within one region the LLM may offer several variants of which at most one is adoptable. DAGSmith selects the subset that maximizes total delta subject to these constraints, a small integer program an off-the-shelf solver dispatches exactly. Because each delta was measured under the candidate’s own materialization, DAGSmith then retunes the combined project once more, since the best assignment for the chosen combination need not match any single candidate’s.

3.4 Frequency-Aware Optimization

This stage takes up opportunity (6), frequency-aware optimization. The stages so far treat each node’s slot time as a fixed per-execution cost. In a recurring pipeline that is misleading: a node’s real contribution is its per-run cost multiplied by how often it must actually run. Treating cost as per-execution therefore overvalues savings on rarely-run models, undervalues them on frequently-run ones, and hides a class of refactoring entirely. DAGSmith adds a frequency dimension to correct this. Effective frequencies are derived from

the declared schedule at sink nodes and the observed update rates at source tables, propagated through the DAG, and applied at three layers; the first two leave the DAG structure intact, while the third feeds a new class of candidates back to candidate identification (Section 3.1).

Layer 1: Frequency as cost weight. Materialization tuning and selection (Section 3.3) minimize cost as if every model ran at the same rate. DAGSmith reweights their cost-model inputs by each node’s effective frequency before the decisions are made; nothing else about them changes.

Layer 2: Reconciling scheduled and effective frequency. A model’s effective frequency is the rate at which it produces *new* output, the slower of how often its consumers need fresh output (f_j^{demand}) and how often its inputs actually change (f_j^{data}):

$$f_j = \min(f_j^{\text{demand}}, f_j^{\text{data}}).$$

Where the schedule fires faster than f_j , the model re-executes and reproduces the output it produced last time. DAGSmith flags such over-scheduled models and reduces their schedule to f_j . This changes neither SQL nor materialization, only the schedule, yet every redundant execution it removes is eliminated outright.

Layer 3: Frequency-driven splitting hints. The first two layers leave the DAG structure intact; the third changes it. When a model has parents that change at different rates, it inherits the fastest, so any work inside it that depends only on slow-changing parents is recomputed at the fast rate even though its inputs have not changed. DAGSmith identifies such models and emits the location to candidate identification as a hint, alongside the reuse and prune signals. The LLM then proposes a frequency split of the kind in Example 3: the slow-side work is extracted into its own slow-cadence model and the original becomes a lightweight join, after which the candidate flows through the same filtering, rewriting, verification, and selection as any other.

4 EVALUATION

We evaluate DAGSmith on real dbt projects, organized around three questions:

- (1) How does DAGSmith compare to existing baselines for SQL pipeline optimization, and what classes of optimization account for the gap?
- (2) How much does each component of DAGSmith’s architecture contribute to the overall gain? We ablate the three optimization dimensions (refactoring, materialization, and frequency) and the dataflow-driven targeting that feeds the LLM stage.
- (3) Is the underlying machinery sound and affordable? We measure the accuracy of the cost model that drives materialization decisions, the effectiveness of the equivalence-verification loop at catching unsafe rewrites, and the LLM cost of running the system end-to-end.

Summary of results. [PLACEHOLDER] will come back later. Note that we should explicitly say the cost is the cost on bigquery, not the overall cost

Table 2: Frequency profile averages per project. Each row is the fraction of root and leaf models at each cadence, averaged across three profiles.

Project	Roots			Leaves		
	wk	day	hr	wk	day	hr
Tuva	0.18	0.57	0.25	0.07	0.20	0.73
Stripe	0.13	0.60	0.27	0.05	0.52	0.43

4.1 Experimental Setup

Workloads. We evaluate on two real dbt projects. *Tuva* is an open-source dbt package for healthcare data transformation. The full project comprises 842 SQL models organized across 17 modules. We use the claims preprocessing module, which contains 255 models (about 30% of the project) and is the subset whose models exhibit substantial computational complexity. We run it on a synthetic patient population of 10K, with the largest source table at 505 MB. *Stripe* is a payments-analytics project of 65 models, already factored by hand into intermediate stages so that the easy syntactic wins have been taken. We run it with the largest source table at 199 MB. The two projects together cover the range we want to evaluate: a large project that has accumulated cross-model patterns from multiple analysts, and a smaller, already-factored project where remaining gains must come from materialization, frequency, or semantic restructuring beyond syntactic deduplication.

Testbed. Experiments run on BigQuery with a 100-slot reservation on the Standard edition under capacity-based pricing. **lin: Mention that our technique can optimize SQL pipelines in general. We just choose BigQuery and dbt as these are the systems that our open source projects use.** We report slot-time as the primary cost metric, since slot-time is the billing unit under capacity pricing, and elapsed time as a secondary latency metric. Each reported number is the median of three runs. To simulate realistic production schedules, we use three frequency profiles per project that assign root and leaf models to hourly, daily, or weekly cadences, with interior models derived by propagation. Table 2 summarizes the average cadence mix per project; all frequency-weighted results in this section are averaged across the three profiles.

Cost metrics. We report two views of cost. *Single-run cost* is the cost of executing the full DAG once. *Frequency-weighted cost* weights each model’s per-execution cost by the rate at which it actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project’s billed cost over a period. Refactoring and materialization affect both metrics; frequency optimization affects only the frequency-weighted metric. Improvements are reported as percent reduction relative to the original project.

Baselines. We compare against three baselines spanning the design space of SQL pipeline optimization.

LearnedRewrite [?] **lin: broken citation.** is a rule-based per-query rewriter. It applies a portfolio of equivalence-preserving transformations to each model’s compiled SQL independently, guided by a learned policy that selects which rules to apply and in what order. It is competitive on standalone analytical queries but has no view of the surrounding DAG.

Table 3: Improvement over the original project (% reduction; higher is better). Medians of three runs on a 100-slot BigQuery reservation.

	Single-run		Freq-weighted	
	Elapsed	Slot	Elapsed	Slot
<i>Tuva (255 models)</i>				
LearnedRewrite	−19.6	−2.5	−20.2	−3.0
GenRewrite	9.7	17.7	21.5	15.1
CSE	12.2	34.6	27.7	37.9
DAGSmith	30.9	51.1	42.6	67.7
<i>Stripe (65 models)</i>				
LearnedRewrite	2.6	2.7	−0.5	4.5
GenRewrite	7.6	7.1	1.1	9.3
CSE	−3.2	−5.6	−3.1	−0.6
DAGSmith	40.0	47.4	36.1	54.8

GenRewrite [13] is an LLM-based per-query rewriter. It introduces natural-language rewrite rules that transfer knowledge across queries and a counterexample-guided loop that iteratively corrects syntactic and semantic errors in the rewrites. Like *LearnedRewrite*, it operates one model at a time.

CSE is a cross-model common subexpression exploitation baseline patterned after Silva-Larson-Zhou [?], **lin: broken citation.** which detects subexpressions shared across a SCOPE script’s stages and materializes each once, re-optimized at the query optimizer’s memo. The memo step is not portable, since managed warehouses do not expose their optimizer’s internal state to external tools. We adapt the approach by detecting shared subexpressions at the AST level and selecting which to materialize heuristically. *CSE* is the strongest available position for a syntactic, cross-model approach on a managed warehouse.

4.2 Comparison with Baselines

Table 3 reports the improvement of each method over the original project on *Tuva* and *Stripe*. *DAGSmith* leads on every metric on both projects. The single-run slot column captures the cost of one DAG execution and is affected only by refactoring and materialization. The frequency-weighted slot column weights each model’s per-execution cost by the rate at which the model actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project’s billed cost over a period. The gap between the two columns is the contribution of frequency optimization, which affects only the frequency-weighted view.

lin: I think we can tighten the discussions here if we’re short on space (which I think we probably are). Right now, the discussions of each baseline and each optimization DAGSmith did are quite detailed. I think we can cut each paragraph by 20%-30% of we want to save space.

Each baseline falls short for a different structural reason. We name the limit in each case, then summarize the classes of optimization that account for the gap above the strongest baseline.

LearnedRewrite: rules tuned for standalone queries do not generalize to dbt models. *LearnedRewrite* applies equivalence-preserving rewrite rules independently to each model. On *Tuva*, the

one rule that fires at scale converts `SELECT DISTINCT . . . JOIN` into a join of two pre-deduplicated inputs. It wins on models whose join keys are nearly unique (the pre-dedup is cheap and shrinks the join) and loses on models whose join keys are coarse (the pre-dedup forces a full grouping pass of a wide table and the resulting plan is worse than what the warehouse planner would have chosen). The wins and losses cancel. On Stripe, an already-factored project, the syntactic shape that triggers the rule is largely absent and the rule barely fires. The headlines on both projects are essentially flat. Rule-based per-query rewriting has nothing left to do once a planner has handled the easy cases and cross-model structure is out of reach.

GenRewrite: per-model LLM reach helps where there is slack, hurts where there is not. GenRewrite finds real per-model algorithmic rewrites that a rule-based system cannot: dynamic predicate derivation through inequality joins, functional-dependency-aware splitting of wide enrichment joins, replacement of $O(n^2)$ self-joins with $O(n)$ window-function plans. On Tuva these wins account for most of its 17.7% slot reduction. On Stripe the same approach gains far less (+7.1% slot, against +17.7% on Tuva), and a single-run reading suggests it can be net negative on a project this small. The LLM rewrites the project’s largest models with column projections the planner was already applying for free, and disrupts plans the warehouse had already optimized; it has no feedback signal telling it that nothing needs to change. The lesson generalizes: an LLM rewriter operating one model at a time helps where there is algorithmic slack a planner cannot resolve and hurts where there is none.

CSE: syntactic matching sees duplication, not equivalence. CSE detects subexpressions shared across models at the AST level and is the only baseline with cross-model scope. It is strong where the same SQL appears in multiple places, collapsing Tuva’s largest classifier family from 6,135 s to 613 s on syntactic match alone, and weak everywhere else. Models that compute the same operation over different inputs, with different join orders, filter values, and upstream references, share the work in substance but not in SQL; CSE cannot match across them. On Stripe, where a human engineer has already taken the easy syntactic wins, CSE has almost nothing to do.

DAGSmith: four patterns that the baselines structurally cannot reach. The classes of optimization that produce DAGSmith’s gap above the strongest baseline fall into four recurring patterns. The first two are forms of cross-model refactoring that go beyond syntactic matching; the next two exercise dimensions outside compiled SQL altogether. Each is illustrated with one concrete instance from the workloads.

(1) *Semantic-equivalence factorization across dissimilar siblings.* Models that compute the same logical operation over different inputs are folded into one shared intermediate, even when their SQL has no syntactic overlap. On Tuva, six inpatient encounter-rollup models compute the same per-encounter aggregations over different encounter types; DAGSmith synthesizes a single rollup that carries the encounter type as a column and replaces all six bodies with lookups against it. The family drops from 914 s to 211 s, where CSE recovers only 127 s of that gap. This is the regime where

syntactic AST matching cannot reach but semantic equivalence is clear once the LLM stage reads each model.

(2) *Cross-model narrowing.* When many consumers each scan a wide table to read a small subset of its columns, the projection is extracted into a narrow intermediate consumed by all of them. The pattern shows up wherever a fan-out exists: many readers, one wide source, narrow per-reader needs. On Tuva, a 14-branch union consumer reads a single column from a wide claim-line table on each branch; once DAGSmith extracts the narrow projection, the consumer drops from 1,420 s to 374 s. The shared work here is column-level; per-query rewriters do not see it and AST matching does not find it because each branch references a different upstream model.

(3) *Materialization revision unlocks planner reach.* When a model is materialized as a table, the warehouse planner sees an opaque scan; when it is materialized as a view, the planner sees the full source SQL and can push predicates, prune columns, and reorder joins across the staging boundary. DAGSmith treats materialization as an optimization variable and revises it across the project. On Stripe, flipping the staging layer of a wide reporting model from tables to views drops the model’s bytes scanned by a third and its slot by a factor of 2.5, on byte-identical model SQL. The downstream savings are produced entirely by the planner, but they exist only because DAGSmith changed the materialization choice; no approach that operates on compiled SQL can reach them.

(4) *Materialization revision tied to refactored structure.* The right materialization for the original project is not the right materialization for the refactored project. Once a refactoring changes which models read from which, the build cost of an upstream table can stop being worth paying. On Tuva, six voting and normalize stages were materialized as tables in the original project, each feeding a single downstream consumer; once DAGSmith refactors the consumers, the tuner flips the upstream stages to views, and the family’s slot drops by 24%. CSE, which applies the original project’s materialization choices to its rewrites, regresses on the same family. This is direct evidence that refactoring and materialization must be tuned jointly: neither pass in isolation finds the configuration that the joint pass does.

Frequency optimization is the fifth dimension and contributes the gap between the single-run and frequency-weighted columns of Table 3: DAGSmith reduces the firing rate of models whose inputs change less often than they execute, eliminating redundant runs outright. Like materialization, it is invisible to any approach that operates on compiled SQL.

Tuva and Stripe exercise these patterns in different proportions. Tuva’s gap above the strongest baseline is dominated by patterns 1 and 2 (cross-model refactoring); Stripe’s is dominated by pattern 3 (materialization revision). The same pipeline produces both outcomes because the optimizer chooses what to apply to the project in front of it, rather than committing to a single mechanism upfront. The next subsection quantifies the contributions component by component.

Table 4: Tuva: dimension ablation (% reduction over the original project, averaged across the three Tuva profiles in Table 2). R = refactoring, M = materialization tuning, F = frequency optimization. **lin: Why we bold the R+M+F number for the elapsed time? It’s not the best, right? That’s actually not a super big problem, as we don’t need to be better on every single number. But make sure you highlight the correct number, and in the discussion, make sure you explain this (why full is not the best on elapsed time) and your statements are consistent with the numbers here.**

Variant	Elapsed	Slot
original	–	–
F only	13.2	11.0
M only	34.7	37.0
R only	40.0	49.1
M + F	31.0	54.0
R + M	43.1	54.7
R + M + F (full)	42.6	67.7

4.3 Component Ablation

Section 4.2 described the *classes of optimization* responsible for DAGSmith’s gap above the baselines. This subsection asks a different question: among DAGSmith’s own architectural components, how much does each contribute, and how do they interact? We ablate along two axes: the three optimization dimensions of refactoring (R), materialization tuning (M), and frequency optimization (F); and the dataflow analysis that targets the LLM stage. The ablation is performed on Tuva alone, because Tuva is the larger and more structurally varied of the two projects and exercises all three dimensions meaningfully; Stripe is small enough that its single-run signal sits close to the run-to-run noise band, making fine-grained dimension comparisons unreliable.

4.3.1 Three Optimization Dimensions. Table 4 reports the improvement of each subset of $\{R, M, F\}$ over the original Tuva project.

The numerics admit four observations, each one reflecting a property of how the dimensions are designed to interact.

Slot reductions exceed elapsed reductions across every variant. Slot measures the total CPU-time the project consumes; elapsed measures the wall-clock time from first model to last. DAGSmith removes work parallel to the critical path more aggressively than work on the critical path itself: a refactoring that consolidates many independent sibling models into one shared intermediate saves slot proportional to the number of siblings but saves elapsed only the slowest sibling’s share. The slot-vs-elapsed gap is largest for R-only (49.1% vs 40.0%), where this effect is most direct, and remains visible across the rest of the table.

R-only is closer to R+M than the headline suggests, because R is not pure refactoring. When DAGSmith’s refactoring stage introduces a new model, it also assigns that model an optimal materialization as part of producing the candidate (§??). The R-only variant therefore already tunes the materialization of newly created models; what M adds on top is project-wide retuning of pre-existing models, which is a smaller marginal change. The gap between R-only (49.1%) and R+M (54.7%) is correspondingly modest. M-only

(37.0%) is lower than R-only because it cannot create models at all, so it can only revise materialization on the original DAG.

Refactoring and materialization are coupled. R+M (54.7%) exceeds the larger of the two singletons (R: 49.1%) by 5.6 points. The refactored DAG admits a different optimal materialization than the original DAG, and the joint pass finds savings that neither stage produces in isolation. The Section 4.2 argument that refactoring and materialization decisions must be made jointly is made concrete here: pattern 4 of Section 4.2 contributes to R+M but not to R or M alone.

Frequency’s increment depends on what came before it. F alone delivers 11.0%; F adds 17.0 points on top of M (54.0 vs 37.0); F adds 13.0 points on top of R+M (67.7 vs 54.7). F’s reach is bounded by the over-scheduled work present in the project, and that pool itself depends on the prior dimensions. Refactoring can change which models are over-scheduled (a refactoring that splits a fast-cadence model into a slow rollup and a fast lookup moves work onto the appropriate cadence and removes it from F’s surface), while materialization tuning leaves the over-schedule pool largely intact. F therefore has more to remove on top of M than on top of R+M. The non-collapsing increment confirms that F is structurally orthogonal to R and M and not a redundant pass; the size of the increment confirms that the three dimensions interact in non-trivial ways.

4.3.2 Dataflow Analysis. **lin: Is this subsection what you’re planning to do or what’s done? I’m confused... Where are the results/tables?**

The dimension ablation above takes DAGSmith’s candidate set as given. This experiment instead isolates the contribution of the dataflow analysis that produces that set (§3.1): does structural targeting direct the LLM stage to more productive regions of the DAG than alternatives that do not use it? The central design principle is that the LLM-call budget must be held fixed across all arms. Otherwise the comparison would merely reflect that more LLM invocations uncover more refactorings, rather than whether dataflow analysis spends a fixed budget more effectively. In the standard pipeline, dataflow analysis selects at most [20] subgraphs on Tuva, each capped at [60] nodes, and the LLM is given [3] diagnostic-and-correction attempts per subgraph. We hold this per-subgraph attempt budget fixed across every arm and compare against two alternatives.

Whole-DAG refactoring. The first alternative supplies the entire project to the LLM under the same 3-attempt budget. This arm tests whether restricting the LLM’s attention to a bounded region is necessary at all. On Tuva, the whole DAG fits within the context window (134k input tokens), and the analysis phase succeeds: the LLM identifies most of the opportunities that our pipeline surfaces. Implementation then fails for three compounding reasons. First, length: the response must restate every rewritten model in full, growing long enough that the LLM fails to finish and silently truncates, leaving dangling references. The truncated output is not a runnable project, so this arm yields no valid DAG to measure; we therefore report its behavior rather than a cost. Second, interdependence: the more models a single attempt rewrites, the more cross-model references it must keep consistent, and a fix to one model breaks an assumption in another. Third, non-convergence: because every broad attempt breaks somewhere, the correction loop

retreats in scope, with the three attempts rewriting successively fewer models, 51, then 35, then 8. This exposes a dilemma whole-DAG refactoring cannot escape: a broad rewrite captures most gain but touches too many interdependent models to verify, while a rewrite narrow enough to verify captures only a fraction of the gain. With no way to decompose the project, the LLM retreats to the safe, low-gain end. Dataflow analysis resolves the dilemma: each subgraph is small enough to verify, yet the pipeline still reaches the high-value regions by examining them one at a time.

Random subgraphs. The second alternative keeps everything fixed except the selection rule: the same subgraph count, the same size cap, and the same downstream refactoring procedure, with dataflow-guided selection replaced by random selection. We draw a model uniformly at random and take its parents, children, and siblings as a subgraph, repeating until the count matches the standard pipeline and discarding any neighborhood over the node cap. This isolates the contribution of *where* the pipeline chooses to look. Random selection reaches a single-run slot reduction of 43.1% and a frequency-weighted reduction of 58.0%, against 51.1% and 67.7% for dataflow-guided selection. The gap reflects a mismatch between two structures. An edge-based subgraph groups a model with its dependency neighbors, but the models worth refactoring together are not its dependency neighbors: they are models elsewhere in the project that compute the same pattern. Two sibling pipelines may each apply the same expensive transformation to different inputs without sharing a single edge, so no edge-based neighborhood contains both, and widening the neighborhood with more hops only adds dependency-adjacent models that are irrelevant to the refactoring. Dataflow-guided selection groups on the structure of the computation rather than on dependency edges, so it places the models that compute the same pattern in one subgraph regardless of where they sit in the DAG.

4.4 System Characterization

lin: The topics seem reasonable to me, but we need to finish this subsection...

This subsection asks whether the machinery that produces the gains above is itself sound and affordable. We measure the accuracy of the cost model that drives materialization and refactoring decisions, the effectiveness of the equivalence-verification loop at catching unsafe rewrites, and the LLM cost of running DAGSmith end-to-end.

4.4.1 Cost Model Accuracy. [PLACEHOLDER] To be filled in. Reports the accuracy of the Huber-regression cardinality and slot-time predictors against measured values on the original and refactored projects.

4.4.2 Equivalence-Verification Effectiveness. [PLACEHOLDER] Reports how the equivalence-verification loop behaves on the refactoring candidates produced for Tuva and Stripe: the number of candidates that pass on the first attempt, the number that require one or more revision rounds, and the number rejected outright. For candidates the loop rejects or revises, we characterize the kinds of unsafe transformations caught (e.g., the secondary-tag drop illustrated in Figure ??). The end-to-end question is whether the loop is

conservative enough to be trusted as the project’s safety net, while not so conservative that real refactorings are discarded.

4.4.3 LLM Cost. [PLACEHOLDER] Reports total LLM tokens consumed and dollar cost of running DAGSmith end-to-end on Tuva and on Stripe, broken down by stage (subgraph analysis and opportunity proposal, proposal filtering, refactoring, equivalence verification, family propagation). Numbers TBD.

5 RELATED WORK

The most closely related lines of work are database query optimization, source-to-source SQL rewriting, multi-query optimization, materialized-view recommendation, workflow scheduling, and data lineage or clone analysis for analytics pipelines. DAGSmith differs from these lines by treating the explicit SQL pipeline DAG itself as the optimization object, allowing it to restructure node boundaries and reason jointly about semantics, materialization, and refresh frequency.

Database query optimizers remain essential but orthogonal: they choose physical plans for one submitted SQL statement. DAGSmith operates before that point, changing the maintained SQL pipeline program that produces those statements. Source-to-source SQL rewriters such as *SlabCity* [10], *GenRewrite* [13], *WeTune* [21], and *R-Bot* [18] also operate before the database optimizer, but they optimize one statement at a time and do not change the dependency graph around it.

Multi-query optimization and common-subexpression sharing reason across multiple statements, but typically over flat workloads and mostly through syntactic overlap. They do not usually exploit output contracts, declared refresh cadences, materialization choices, or stable developer-maintained node boundaries. Materialized-view selection optimizes persistence decisions, but does not usually rewrite the graph to move computation across freshness boundaries or remove intermediate contracts. Workflow systems such as Airflow [2], Dagster [4], Prefect [15], Argo [3], Luigi [14], SQLMesh [19], and Databricks Workflows [5] expose dependencies and schedules, but their optimizers primarily decide ordering, invalidation, or execution policy rather than semantic SQL rewrites. DAGSmith targets the gap between these areas: dependency-aware source-to-source optimization for recurring SQL pipeline DAGs.

6 CONCLUSION

Explicit SQL pipeline DAGs create an optimization setting that is richer than traditional one-query-at-a-time rewriting. Once dependencies, node contracts, materialization choices, and refresh behavior are visible, the optimizer can restructure the pipeline itself rather than merely improve individual statements. DAGSmith is a step toward that goal: it treats dependency-aware pipeline optimization as a joint problem over graph structure, materialization, and frequency, and combines structural analysis with LLM-guided refactoring and verification to explore that space.

The core claim is that recurring SQL pipeline DAGs expose whole-pipeline optimization opportunities that are invisible to query-local rewriting, and exploiting them requires a system designed for dependency-aware reasoning.

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.
- [2] Apache Airflow. 2026. DAGs. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>. Accessed 2026-05-26.
- [3] Argo Workflows. 2026. DAG. <https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/>. Accessed 2026-05-26.
- [4] Dagster Labs. 2026. Software-defined assets. <https://docs.dagster.io/>. Accessed 2026-05-26.
- [5] Databricks. 2026. Configure task dependencies. <https://docs.databricks.com/aws/en/jobs/run-if>. Accessed 2026-05-26.
- [6] Databricks. 2026. What is the medallion lakehouse architecture? <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>. Accessed 2026-05-26.
- [7] dbt Labs. 2024. The next big step forwards for analytics engineering. <https://www.getdbt.com/blog/analytics-engineering-next-step-forwards>. Accessed 2026-05-26.
- [8] dbt Labs. 2024. What is data lineage, and why do you need it? <https://www.getdbt.com/blog/what-is-data-lineage>. Accessed 2026-05-26.
- [9] dbt Labs. 2025. 2025 State of Analytics Engineering Report. <https://www.getdbt.com/resources/reports/state-of-analytics-engineering-2025>. Accessed 2026-05-26.
- [10] Rui Dong, Jie Liu, Yuxuan Zhu, Barzan Mozafari, Xinyu Wang, and Cong Yan. 2023. SlabCity: Whole-Query Optimization using Program Synthesis. <https://par.nsf.gov/servlets/purl/10451715>.
- [11] Tristan Handy. 2026. What, exactly, is dbt? <https://www.getdbt.com/blog/what-exactly-is-dbt>. Accessed 2026-05-30.
- [12] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 205–216. <https://doi.org/10.1145/233269.233333>
- [13] Jie Liu and Barzan Mozafari. 2024. Query Rewriting via Large Language Models. <https://arxiv.org/abs/2403.09060>. CoRR abs/2403.09060 (2024).
- [14] Luigi Contributors. 2026. Building workflows. <https://luigi.readthedocs.io/en/stable/workflows.html>. Accessed 2026-05-26.
- [15] Prefect Technologies. 2026. Tasks. <https://docs.prefect.io/v3/concepts/tasks>. Accessed 2026-05-26.
- [16] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhole. 2000. Efficient and Extensible Algorithms for Multi Query Optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 249–260. <https://doi.org/10.1145/342009.335419>
- [17] Timos K. Sellis. 1988. Multiple-Query Optimization. *ACM Transactions on Database Systems* 13, 1 (1988), 23–52.
- [18] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2024. R-Bot: An LLM-based Query Rewrite System. *CoRR* abs/2412.01661 (2024).
- [19] Tobiko Data. 2026. SQLMesh Overview. <https://sqlmesh.readthedocs.io/en/stable/concepts/overview/>. Accessed 2026-05-26.
- [20] Adrian Vogelsgesang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Muehlbauer, Thomas Neumann, and Manuel Then. 2018. Get Real: How Benchmarks Fail to Represent the Real World. In *Proceedings of the Workshop on Testing Database Systems (DBTest '18)*. Association for Computing Machinery, New York, NY, USA, Article 4, 6 pages. <https://doi.org/10.1145/3209950.3209952>
- [21] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *Proceedings of the 2022 International Conference on Management of Data*. 94–107. <https://doi.org/10.1145/3514221.3526125>